



Validating Big Data Models at Scale

A hybrid diagnostic framework — R for statistical rigor, SQL for in-database scale

Regression · Classification · Clustering

The Hybrid Architecture: R + SQL



R diagnoses

Residuals, ROC curves, silhouette — the statistical depth SQL can't express.



SQL scales

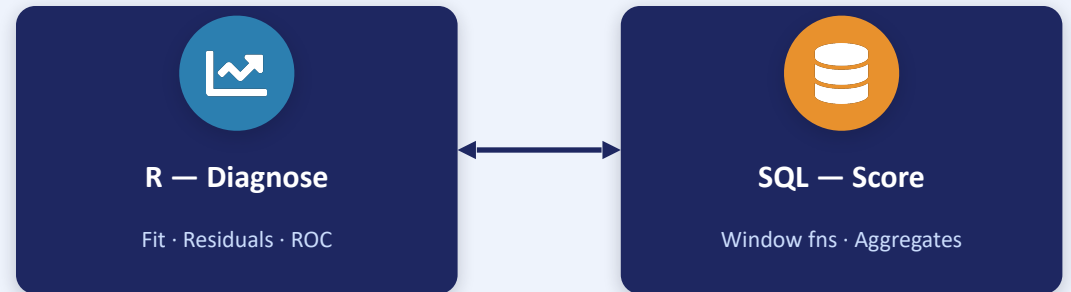
Window functions and aggregates score millions of rows in place.



One protocol

Fit, score, evaluate against a threshold — every model, either engine.

MODEL VALIDATION PIPELINE



coefficients / centroids move — not the data

Rule of thumb: prototype and diagnose on a sample in R; once the metric and threshold are set, run the evaluate step as a query against the full table.

Diagnostic Framework & Key Metrics



R² and Adjusted R²

Share of variance the model actually explains.



RMSE

Typical error, in the same units as the target.



Residual plot

No funnel, no curve — errors scatter randomly around zero.

HOLDOUT R²

0.84

≥ 0.60

HOLDOUT RMSE

17%

≤ 20% mean

RESIDUALS vs. FITTED VALUES



No pattern in the scatter = model isn't missing a trend

In-Database SQL Scaling



Score in place

Predictions land in the warehouse, not in R's memory.



Window functions

AVG() OVER() computes the mean without a second pass.



One query, full table

R² and RMSE recomputed on every row, on a schedule.

Why this matters: the R model only supplies coefficients — scoring millions of restaurants never has to leave the database.

SQL

```
WITH scored AS (  
  SELECT  
    actual_revenue,  
    predicted_revenue,  
    actual_revenue - predicted_revenue  
      AS residual,  
    AVG(actual_revenue) OVER ()  
      AS mean_actual  
  FROM model_scores  
)  
SELECT  
  1 - SUM(POWER(residual, 2))  
    / SUM(POWER(actual_revenue  
      - mean_actual, 2)) AS r_squared,  
  SQRT(AVG(POWER(residual, 2)))  
                                AS rmse  
FROM scored;
```

Diagnostic Framework & Confusion Matrices



Precision

Of the positive calls, how many were right.



Recall

Of the true positives, how many got caught.



F1 score

Harmonic mean — punishes a weak side, either side.

Precision 86.6% · Recall 76.5% · F1 81.2%

Accuracy alone (86%) looked fine — the matrix is what caught the recall gap.

HOLDOUT CONFUSION MATRIX (n = 2,000)

	Predicted: Positive	Predicted: Negative
Actual: Positive	612 True Positive	188 False Negative
Actual: Negative	95 False Positive	1,105 True Negative

Advanced Evaluation & R ROC Curves



AUC

Separability across every possible cutoff, not just one.



TPR vs. FPR

Plotted at each threshold to trace the ROC curve.



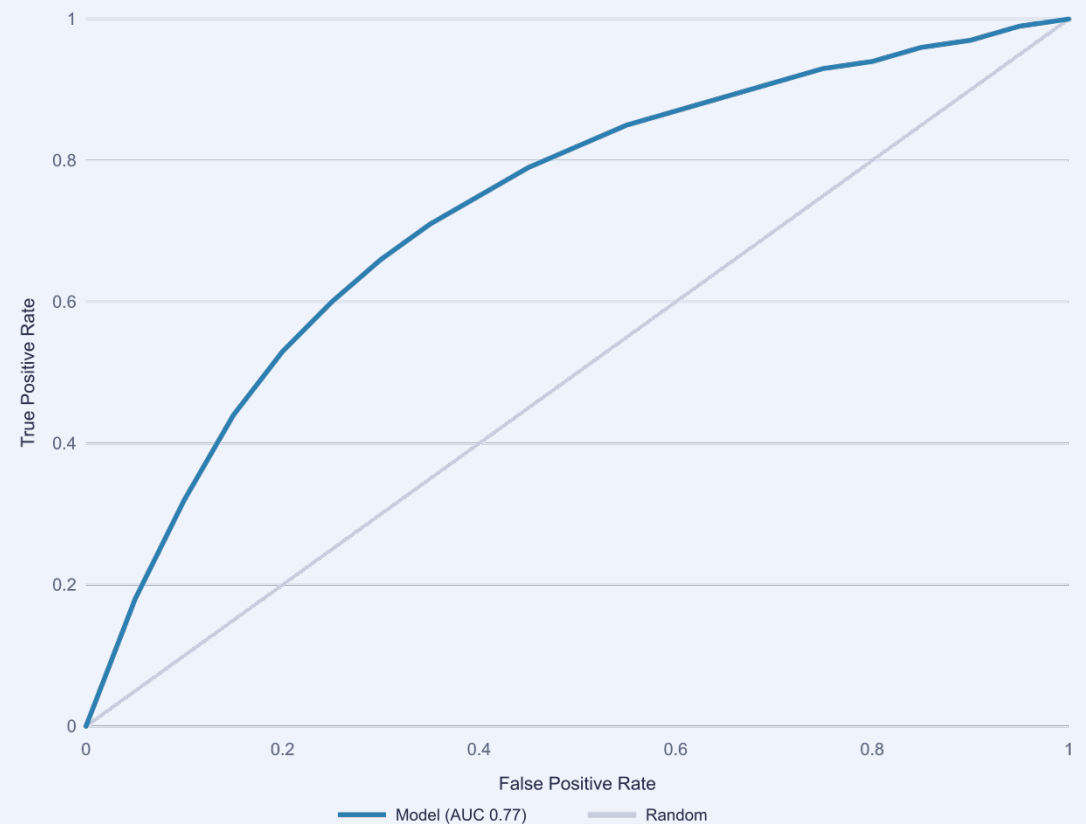
Above the diagonal

Distance from the diagonal is real signal.

R

```
roc_obj <- roc(holdout$actual,
               holdout$pred_prob)
plot(roc_obj, print.auc = TRUE,
     col = "#2C7FB0", lwd = 2)
```

ROC CURVE — HOLDOUT SET



Validation Framework: WCSS vs. BCSS



WCSS

Within-cluster spread — smaller means tighter clusters.



BCSS

Between-cluster spread — bigger means cleaner separation.



Silhouette score

Combines both into one -1 to 1 verdict per point.

SILHOUETTE, BEST k (k = 3)

0.42

≥ 0.50 target

WCSS BY CLUSTER COUNT (ELBOW)



Diminishing returns past k=3 — but silhouette says the segments still overlap.

In-Database Centroid Distance



Centroids as a lookup

A handful of rows — cheap to join against.



CROSS JOIN + window

Every row compared to every centroid, ranked.



ROW_NUMBER() = 1

Nearest centroid, and its distance, in one pass.

Why this matters: WCSS for millions of rows becomes one grouped SUM() over this same result set.

SQL

```
WITH distances AS (  
  SELECT  
    r.restaurant_id, c.cluster_id,  
    SQRT(  
      POWER(r.cost_scaled  
        - c.cost_centroid, 2) +  
      POWER(r.rating_scaled  
        - c.rating_centroid, 2)  
    ) AS dist,  
    ROW_NUMBER() OVER (  
      PARTITION BY r.restaurant_id  
      ORDER BY dist  
    ) AS rn  
  FROM restaurants r  
  CROSS JOIN centroids c  
)  
SELECT restaurant_id, cluster_id,  
       dist AS wcss_contribution  
FROM distances WHERE rn = 1;
```


Big Data Diagnostics Best Practices



Threshold governance

Set the bar before scoring — a metric with no threshold is an opinion.



Drift monitoring

Recompute R^2 and AUC on a rolling holdout, on a schedule.



Compute placement

Diagnose in R, score in SQL — move the model, not the data.



Reproducibility

Version coefficients and thresholds together, same as code.

CONCLUSION

Next Steps

- 1 Adopt fit → score → evaluate for every model, every time
- 2 Prototype diagnostics in R, then push scoring into SQL
- 3 Re-run every metric on a live holdout, not once at launch

Questions?

